Simulation Report

Event Trigger Polynomial Control

Abstract

We tried to simulate various systems inspiring from the paper and from the "modeling of system" lecture taught in the class on 16th April. First we considered a simple one-dimensional system and tracked its reference trajectory. After our first simulation, we felt confident enough to apply the 2nd degree polynomial control to a 2D circular trajectory tracking system followed by "Figure 8" trajectory tracking in 2D. Then we shifted to trajectory tracking of a 3-Dimensional system which was also forming a Figure 8-like path when seen from a 2D angle. After these simulations, we tried to perform Lyapunov stability on 2D circular trajectory tracking. We also simulated ETPC, ETC, and UU boundedness.

One Dimension System

Defining the 1D system

Imagine a straight cart on a frictionless track which is in 1D. Let the state vector be defined by position $p(t)$ and velocity $v(t)$:

$$x(t) = \begin{bmatrix} p(t) \\ v(t) \end{bmatrix} \tag{1}$$

Let the input acceleration be $u(t) = a(t)$. The linear time-invariant (LTI) dynamics can be expressed as:

$$\dot{x}(t) = Ax(t) + Bu(t) \tag{2}$$

which expands to:

$$\begin{bmatrix} \dot{p} \\ \dot{v} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} p \\ v \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u \tag{3}$$

References

The reference trajectories are given as follows:

$$\text{Position:} \quad p_r(t) = \cos(t) \tag{4}$$

$$\text{Velocity:} \quad v_r(t) = -\sin(t) \tag{5}$$

$$\text{Acceleration:} \quad u_r(t) = -\cos(t) \tag{6}$$

Tracking Error

The tracking error $e(t)$ is defined as the difference between the actual state and the reference state:

$$e(t) = x(t) - x_r(t) \tag{7}$$

We will use a Linear Quadratic Regulator (LQR) to find the gain matrix K. The cost function is:

$$J = \int_0^\infty (x^T Q x + u^T R u)\, dt \tag{8}$$

which will further result in developing an ideal control law, i.e., $u = -Ke(t)$.

Thus, the ideal control input is:

$$u_{\text{ideal}} = u_r(t) - Ke(t) \tag{9}$$

Note: We will use MATLAB to find gain K and simulate our event-triggered polynomial control.

The stable closed-loop error dynamics are given by:

$$\dot{e}(t) = (A - BK)e(t) \tag{10}$$

Defining the Event Triggered Polynomial Control (2nd Degree)

For a 2nd degree polynomial control, we will use 3 polynomial coefficients. Based on the reference inputs:

$$u_r = c_0 \quad \text{(Acceleration)} = -\cos(t) \tag{11}$$
$$\dot{u}_r = c_1 \quad \text{(Jerk)} = \sin(t) \tag{12}$$
$$\ddot{u}_r(t) = c_2 \quad \text{(Snap)} = \cos(t) \tag{13}$$

Suppose we are communicating at time instant (t_k). Our second degree polynomial coefficients will look like:

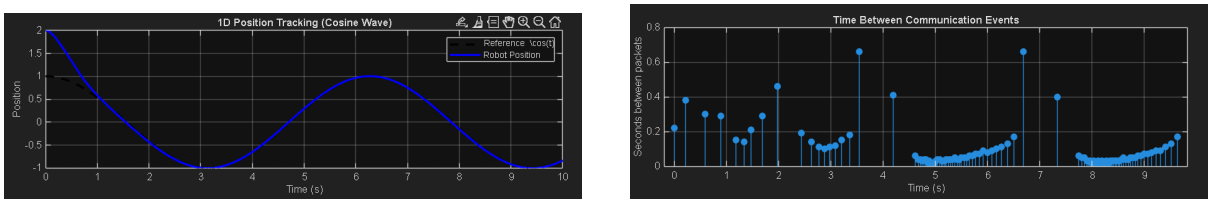
$$c_0 = u_r(t_k) - Ke(t_k) \tag{14}$$
$$\begin{aligned} c_1 &= \dot{u}_r(t_k) - K\dot{e}(t_k) \\ &= \dot{u}_r(t_k) - K(A - BK)e(t_k) \end{aligned} \tag{15}$$
$$\begin{aligned} c_2 &= \ddot{u}_r(t_k) - K\ddot{e}(t_k) \\ &= \ddot{u}_r(t_k) - K(A - BK)^2 e(t_k) \end{aligned} \tag{16}$$

Event Trigger 2-Degree Polynomial Control Input

Thus, our event-triggered 2-degree polynomial control input will look like:

$$u_{\text{applied}} = c_0 + c_1(t - t_k) + c_2(t - t_k)^2 \tag{17}$$

Simulation Results



(a) Position tracking performance over time.

(b) Event trigger points at different times.

Figure 1: 1D Simulation results comparing the reference trajectory with the actual state using event-triggered control.

2D System: Circular Trajectory Control

We want our robot to follow a circular trajectory on the X-Y plane. For this, our X-axis tracks a cosine wave and our Y-axis tracks a sine wave. Let the reference trajectory state vector be defined by position and velocity components for both axes:

$$x_r(t) = \begin{bmatrix} p_x(t) \\ v_x(t) \\ p_y(t) \\ v_y(t) \end{bmatrix} = \begin{bmatrix} \cos(t) \\ -\sin(t) \\ \sin(t) \\ \cos(t) \end{bmatrix} \tag{18}$$

Higher Order Derivatives

The reference acceleration vector $u_r(t) = a_{\text{ref}}(t)$, required jerk vector $\dot{u}_r(t) = j_{\text{ref}}(t)$, and snap vector $\ddot{u}_r(t) = s_{\text{ref}}(t)$ are given as follows:

$$u_r(t) = \begin{bmatrix} -\cos(t) \\ -\sin(t) \end{bmatrix} \tag{19}$$

$$\dot{u}_r(t) = \begin{bmatrix} \sin(t) \\ -\cos(t) \end{bmatrix} \tag{20}$$

$$\ddot{u}_r(t) = \begin{bmatrix} \cos(t) \\ \sin(t) \end{bmatrix} \tag{21}$$

Defining the error as 1D

Just like in the 1D system, we will use a Gain matrix K to define the ideal control law:

$$u_{\text{ideal}} = u_r(t) - Ke(t) \quad \text{(where } K = \text{Gain matrix)} \tag{22}$$

The stable closed-loop error dynamics follow:

$$\dot{e}(t) = (A - BK)e(t) \tag{23}$$

2D Event Triggered Polynomial Control (Taylor Series)

Note: This time we are using the standard Taylor Series expansion for the 2nd degree polynomial, applied to the control input vector update at t_k:

$$u(t) = u(t_k) + \dot{u}(t_k)(t - t_k) + \frac{1}{2}\ddot{u}(t_k)(t - t_k)^2 \tag{24}$$

Using the identical derivative logic as the 1D derivation (now vectorized):

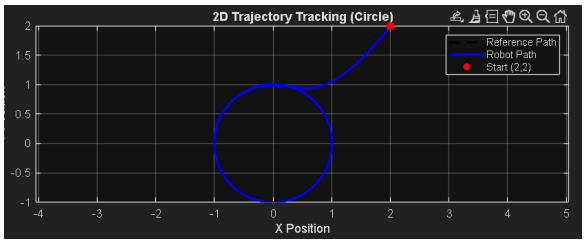
$$u_{\text{ideal}}(t_k) = u_r(t_k) - Ke(t_k) \tag{25}$$
$$\dot{u}_{\text{ideal}}(t_k) = \dot{u}_r(t_k) - K\dot{e}(t_k) \tag{26}$$
$$\ddot{u}_{\text{ideal}}(t_k) = \ddot{u}_r(t_k) - K\ddot{e}(t_k) \tag{27}$$

Event Triggered Applied 2D Polynomial Control

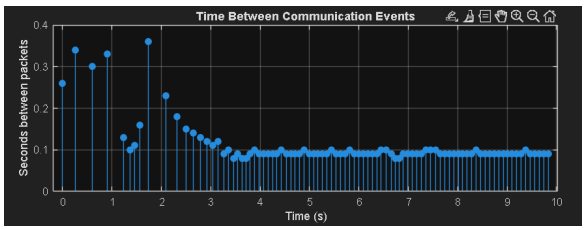
Thus, our applied 2nd-degree polynomial control input vector for the 2D circular trajectory, updated at communication instants t_k, is:

$$u_{\text{app}}(t) = u_{\text{ideal}}(t_k) + \dot{u}_{\text{ideal}}(t_k)(t - t_k) + \frac{1}{2}\ddot{u}_{\text{ideal}}(t_k)(t - t_k)^2 \tag{28}$$

2D Circular Simulation Results



(a) 2D Circular Path tracking.



(b) Event triggers for 2D system.

Differences from 1D (What we did before)

Here are the key mathematical differences noted from our previous 1D setup:

- Now we tracked 2 directions, i.e., X and Y.
- The K gain matrix went from 1×2 to 2×4.

2D System: Figure 8 Trajectory (Lissajous Curve)

For this simulation, we track a Figure 8 trajectory (also known as a Lissajous curve). Basically, it makes the robot draw an "8" on the ground. Unlike the circular trajectory which uses a sine and cosine combination, both of the axes here are controlled by sine waves with different frequencies.

Let the reference position $p_r(t)$ and velocity $v_r(t)$ vectors be defined as:

$$p_r(t) = \begin{bmatrix} \sin(t) \\ \sin(2t) \end{bmatrix}, \quad v_r(t) = \begin{bmatrix} \cos(t) \\ 2\cos(2t) \end{bmatrix} \tag{29}$$

Higher Order Derivatives

The reference acceleration $u_r(t)$, required jerk $\dot{u}_r(t)$, and snap $\ddot{u}_r(t)$ vectors follow standard differentiation:

$$\text{Acceleration:} \quad u_r(t) = \begin{bmatrix} -\sin(t) \\ -4\sin(2t) \end{bmatrix} \tag{30}$$

$$\text{Jerk:} \quad \dot{u}_r(t) = \begin{bmatrix} -\cos(t) \\ -8\cos(2t) \end{bmatrix} \tag{31}$$

$$\text{Snap:} \quad \ddot{u}_r(t) = \begin{bmatrix} \sin(t) \\ 16\sin(2t) \end{bmatrix} \tag{32}$$

2nd Degree Polynomial Equation Formulation

For this trajectory, we return to the polynomial coefficient derivation (c_0, c_1, c_2) rather than the standard Taylor expansion fractions. Assuming communication at time instant t_k, and using our gain matrix K, the coefficients are defined as:

$$c_0 = u_r(t_k) - Ke(t_k) \tag{33}$$

$$c_1 = \dot{u}_r(t_k) - K(A - BK)e(t_k) \tag{34}$$

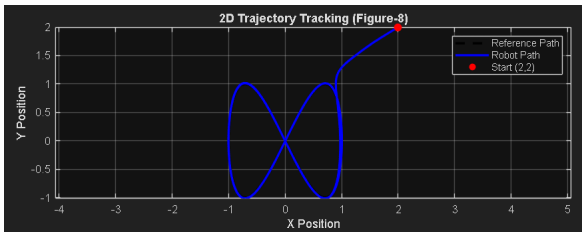
$$c_2 = \ddot{u}_r(t_k) - K(A - BK)^2 e(t_k) \tag{35}$$

Figure 8 Trajectory Polynomial Control

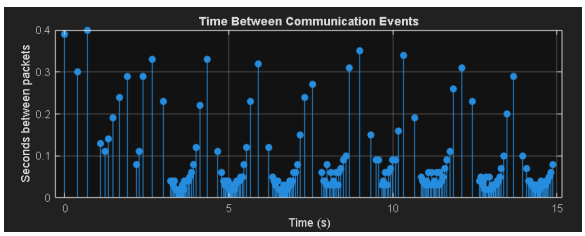
Thus, the applied control input for the 2D Figure 8 trajectory, updated at t_k, is:

$$u_{\text{app}}(t) = c_0 + c_1(t - t_k) + c_2(t - t_k)^2 \tag{36}$$

Figure 8 Simulation Results



(a) Figure 8 Trajectory tracking.



(b) Event triggers for Figure 8.

3D System: Figure 8 Trajectory

To extend our control to 3D space, we expand our state and input vectors to include the Z-axis. The state vector x and input vector u become:

$$x = \begin{bmatrix} p_x & v_x & p_y & v_y & p_z & v_z \end{bmatrix}^T, \quad u = \begin{bmatrix} a_x & a_y & a_z \end{bmatrix}^T \tag{37}$$

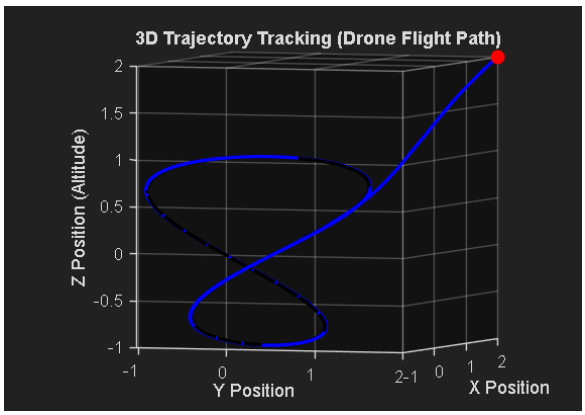
For a 3D Figure 8 trajectory, we define the reference positions as $p_x(t) = \sin(t)$, $p_y(t) = \sin(2t)$, and $p_z(t) = \cos(t)$. The full reference state vector $x_r(t)$ is:

$$x_r(t) = \begin{bmatrix} \sin(t) \\ \cos(t) \\ \sin(2t) \\ 2\cos(2t) \\ \cos(t) \\ -\sin(t) \end{bmatrix} \tag{38}$$

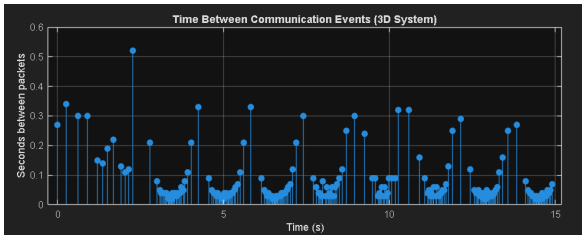
The linear system dynamics $\dot{x}(t) = Ax(t) + Bu(t)$ now use a 6×6 A matrix and a 6×3 B matrix to account for all three independent axes:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{39}$$

3D Simulation Results



(a) 3D Figure 8 Trajectory tracking.



(b) Event triggers for 3D system.

Figure 4: Simulation results tracking a Figure 8 trajectory in 3D space.

Lyapunov Stability Analysis (2D Circular Trajectory)

To ensure the stability of our event-triggered polynomial control (ETPC), we must analyze the perturbed closed-loop error dynamics. In ETPC, the applied control signal (u_{app}) is an approximation of the ideal continuous LQR control (u_{ideal}), separated by an approximation error (ϵ):

$$u_{\text{app}} = u_{\text{ideal}} + \epsilon = u_r - Ke + \epsilon \tag{40}$$

Substituting this back into our error derivative yields the perturbed closed-loop error dynamics, where $(A - BK)e$ provides stabilizing LQR control and $B\epsilon$ represents destabilizing polynomial drift:

$$\dot{e} = Ae + B((u_r - Ke + \epsilon) - u_r) \implies \dot{e} = (A - BK)e + B\epsilon \tag{41}$$

Stability Proof

We define a positive-definite quadratic Lyapunov function candidate, $V(e) = e^T Pe$, where P is a symmetric positive-definite matrix defining our "energy bowl." Differentiating $V(e)$ with respect to time using the product rule gives:

$$\dot{V}(e) = \dot{e}^T Pe + e^T P\dot{e} \tag{42}$$

Substituting our error dynamics equation into this derivative yields:

$$\begin{aligned} \dot{V}(e) &= ((A - BK)e + B\epsilon)^T Pe + e^T P((A - BK)e + B\epsilon) \\ \dot{V}(e) &= e^T (A - BK)^T Pe + \epsilon^T B^T Pe + e^T P(A - BK)e + e^T PB\epsilon \end{aligned} \tag{43}$$

Because P is symmetric ($P^T = P$) and the term $\epsilon^T B^T Pe$ results in a 1×1 scalar (making it equal to its own transpose), we can state $\epsilon^T B^T Pe = e^T PB\epsilon$. Combining these identical terms allows us to factor out e^T and e:

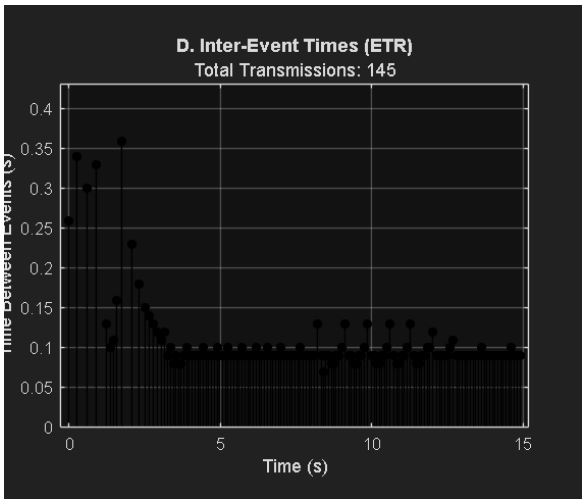
$$\dot{V}(e) = e^T((A - BK)^T P + P(A - BK))e + 2e^T PB\epsilon \tag{44}$$

Finally, we apply the Continuous-Time Algebraic Lyapunov Equation (ALE), which defines $(A - BK)^T P + P(A - BK) = -Q$ for our chosen state penalty matrix Q. Substituting $-Q$ simplifies our derivative to its final form:

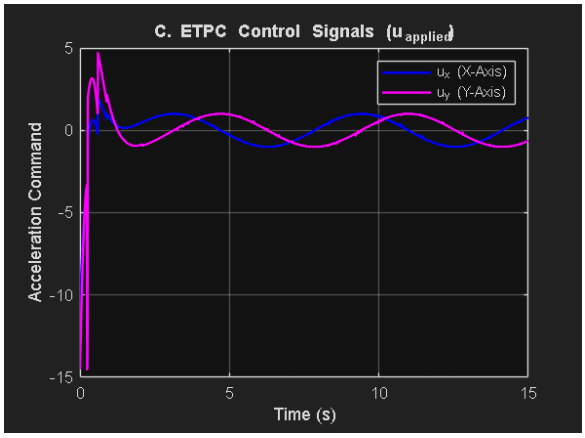
$$\dot{V}(e) = -e^T Qe + 2e^T PB\epsilon \tag{45}$$

The system remains stable as long as the energy dissipation from the LQR controller ($-e^T Qe$) dominates the bounded drift energy added by the event-trigger approximation ($2e^T PB\epsilon$).

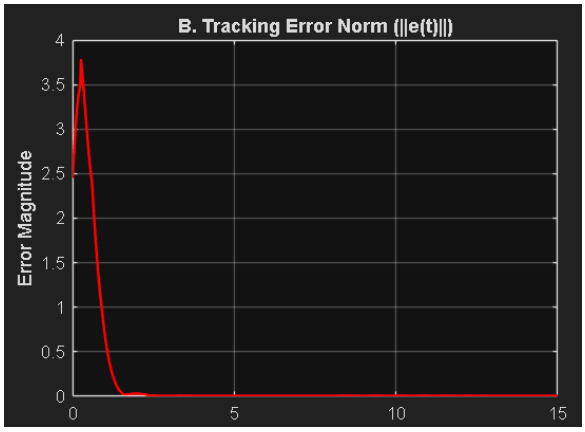
2D Circular Trajectory: Performance Metrics



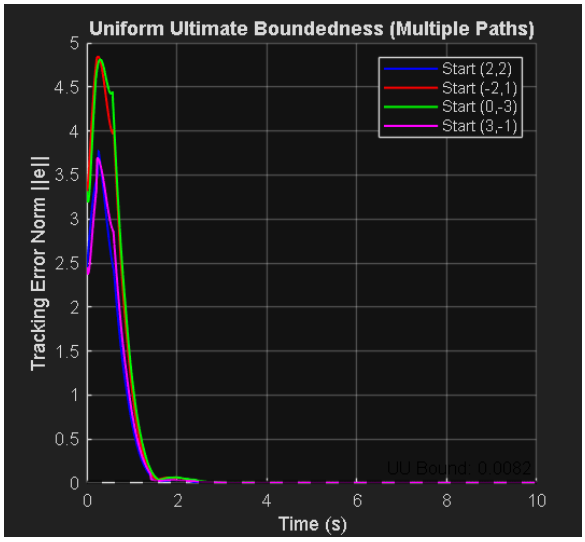
(a) Event Trigger Ratio (ETR)



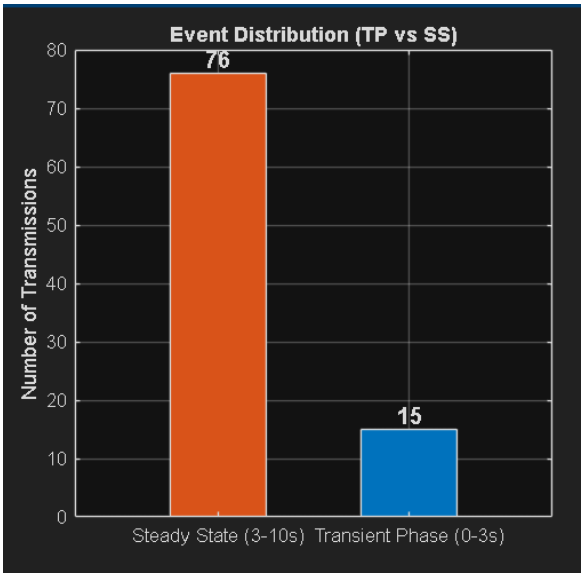
(b) Applied ETPC Signal



(c) Tracking Error Norm



(d) Control Input (u) Boundedness



(e) Event Distribution

Figure 5: Comprehensive performance metrics for the 2D Circular Trajectory tracking.